Live Coding Task — Submission API with Aggregation

You'll build a small backend service and then add live aggregation on top of it. You can use any AI tools you normally work with (ChatGPT, Claude, Copilot, Cursor, etc.) — that's encouraged. We'll ask you to explain your choices as you go, so be ready to walk us through what you built and why.

Language: Go or PHP (your choice)

Part 1 — Build the API

A small REST API with two endpoints:

1. `POST /submissions`

Accepts a form submission as JSON:

json

```
{
  "form_id": "contact-form",
  "idempotency_key": "abc-123",
  "email": "user@example.com",
  "data": { "name": "John", "message": "Hello" }
}
```

It should:

- **Validate the input** — return a `400` error if the email is invalid or `form_id` is missing.
- **Store the submission** in the database.
- **Prevent duplicates:** if the same `idempotency_key` is sent more than once, do **not** create a second record — return the existing one instead.

2. `GET /forms/{form_id}/submissions`

Returns all submissions for a form, **with pagination** (10 per page).

Part 2 — Load test

Once the API works, test it under load and report the results:

- Send **100,000 requests** to `POST /submissions`.
- Make **20,000 of them duplicates** (reuse the same `idempotency_key` values), so 80,000 are unique.
- Report:
- How many submissions were actually stored
- How many duplicates were correctly rejected
- How many errors occurred
- The throughput (requests per second)

You may use any tool you like to generate the load.

Part 3 — Live aggregation with Flink

Now extend your system so that, for every form, it keeps a **live count of submissions, broken down per hour**, updating continuously as new submissions come in.

- Use **Apache Flink** to compute these aggregated counts from the submissions.
- Show the aggregated results working.
- Be ready to explain how your Flink job handles a crash — how it makes sure counts are not lost or double-counted when it restarts.

What we're looking for

- A working API with correct validation and duplicate protection
- Correct load-test results
- Working live aggregation with Flink
- Clean code, and your ability to explain **why** you built things the way you did